WNDRVR
SUPPORT NETWORK

VXWORKS 7 TIME-SENSITIVE NETWORKING PROGRAMMER'S GUIDE, SR0650

Copyright Notice

Copyright © 2022 Wind River Systems, Inc.

All rights reserved. No part of this publication may be reproduced or transmitted in any form or by any means without the prior written permission of Wind River Systems, Inc.

Wind River, Simics, Tornado, and VxWorks are registered trademarks of Wind River Systems, Inc. Helix, Pulsar, Rocket, Titanium Cloud, Titanium Control, Titanium Core, Titanium Edge, Titanium Edge SX, Titanium Server, and the Wind River logo are trademarks of Wind River Systems, Inc. Any third-party trademarks referenced are the property of their respective owners. For further information regarding Wind River trademarks, please see:

www.windriver.com/company/terms/trademark.html

This product may include software licensed to Wind River by third parties. Relevant notices (if any) are provided for your product on the Wind River download and installation portals:

https://delivers.windriver.com/

https://windshare.usa.windriver.com/

Wind River may refer to third-party documentation by listing publications or providing links to third-party websites for informational purposes. Wind River accepts no responsibility for the information provided in such third-party documentation.

Corporate Headquarters

Wind River
500 Wind River Way
Alameda, CA 94501-1153
U.S.A.
Toll free (U.S.A.): +1-800-545-WIND
Telephone: +1-510-748-4100
Facsimile: +1-510-749-2010

For additional contact information, see the Wind River website:

www.windriver.com

For information on how to contact Customer Support, see:

www.windriver.com/support

VxWorks 7 Time-Sensitive Networking Programmer's Guide, SR0650

25 August 2020

1. ABOUT TIME-SENSITIVE NETWORKING

The VxWorks® 7 Time-Sensitive Networking Programmer's Guide describes how Time-Sensitive Networking (TSN) is a technology defined by a large set of sub-standards of the IEEE 802 LAN/MAN family of standards. The IEEE TSN task group is part of the IEEE 802.1 working group that has published several TSN standards.

VxWorks® includes the technology to implement TSN features defined by the following standards:

- IEEE standard 802.1AS-2011—*IEEE Standard for Local and Metropolitan Area Networks - Timing and Synchronization for Time-Sensitive Applications in Bridged Local Area Networks*
- IEEE standard 802.1Qbv-2015—*IEEE Standard for Local and metropolitan area networks - Bridges and Bridged Networks - Amendment 25: Enhancements for Scheduled Traffic*

For more information, see http://www.ieee802.org.

The purpose of the TSN standards is to enable deterministic real-time communication over Ethernet. The two main components required to achieve this are as follows:

- Time transfer
- Timely delivery

Time transfer, means distributing and sharing the same time base in the TSN network. The IEEE 1588 standard and associated profiles define time transfer. Precision Time Protocol (PTP) synchronizes the clocks of different devices on a network by adhering to the IEEE 1588 standard. The 802.1AS profile defines the specific behavior of PTP for use in TSN.

Timely delivery, means guaranteeing packet delivery on a switched Ethernet network with bounded latency. Timely delivery provides a schedule that determines when a device transmits. No other device can transmit during this time-period. Because devices utilize transmit queues with different priorities, a TSN network can also transmit non-scheduled traffic.

VxWorks allows time-triggered transmitting capability in a network device. A preset schedule determines the launch time of a packet and directs the packet to a specific launch queue in the device. All TSN features require specific hardware capabilities.

VxWorks provides support for the following TSN and PTP hardware and protocols.

Hardware	IEEE1588v2/PTP Standard	802.1AS/gPTP Standard	802.1Qbv Standard	802.1Qbu Standard
Intel i210 NIC	Yes	Yes	Yes (Time-Specific Only)	
Intel i350 NIC	Yes	Yes		
Xilinx ZCU102	Yes	Yes		
NXP LS1028A	Yes (Ethernet Endpoint)	Yes (Ethernet Endpoint)	Yes (Time-Specific and Time-Gated) (Ethernet Endpoint)	Yes (Ethernet Endpoint)
Renesas R-Car H3	Yes			

Refer to the manufacturer’s Ethernet device documentation and the corresponding VxWorks enhanced network device driver information to support TSN on your specific device.

WNDRVR

2. TSN CLOCKS AND TIMERS

Time-Sensitive Networking (TSN) capable devices provide synchronized time in special registers called Precision Time Protocol (PTP) hardware clock (PHC) registers. PHC registers are available for a user application through input output control (IOCTL) operations implemented in the device driver.

VxWorks provides a user friendly application programming interface (API) to access PTP time from the device. For more information see, the **tsnClkLib** library API reference entry. The library implements user access to timer interrupts based on the PHC, provided the device itself supports this capability.

Note: To enable TSN clocks and timers, the TSN and PTP VSB options must be included.

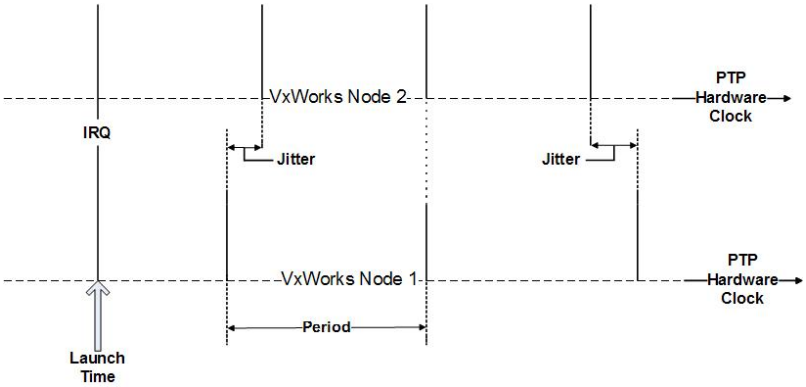
User Access to the PTP Hardware Clock

User applications acquire the identifier to the PHC with the tsnClockIdGet( ) routine and then get the PTP time with the tsnClockTimeGet( ) routine. The example below shows how to find the clock ID and then get the current PTP time into a timespec structure:

```
clockid_t cid;
struct timespec tm;
cid = tsnClockIdGet("gei",0,0);
    if (cid != 0)
    {
    ret = tsnClockTimeGet (cid, &tm);
    }
```

Synchronized Timer Interrupts

Because the PTP software keeps the PHC in sync with other devices it is possible to create timer interrupts that fire synchronously between different devices on the network. User applications may program a timer based on the PHC to fire periodically, or just once at a future time (one-shot). The main advantage of this feature is that the timer expires at the same time on all devices and the connected interrupt routine can create RTOS events that occur synchronously. The figure below shows an example of a periodic TSN timer on two different VxWorks devices:



Note that there will be some inaccuracies between the interrupts (jitter) due to hardware latencies and other sources of error. The interrupts will not drift apart over time because the clock source (PHC) stays in sync. The example below shows how to program a TSN timer to fire every millisecond (1000 Hz):

```
cid = tsnClockIdGet("gei",0,0);
    if (cid != 0)
    {
```

```
        ret = tsnTimerAllocate (cid);
        if (ret == OK)
        {
            ret = tsnClockConnect (cid, (FUNCPTR)usrTsnClockIsr, 0);
            if (ret == OK)
                ret = tsnClockRateSet(cid, 1000);
         }
     }
```

In the example above the user-defined routine, usrTsnClockIsr( ), will be called at every interrupt. Because this routine is called from an interrupt context normal VxWorks API restrictions apply. Use tsnClockEnable( ) to enable the timer. For additional information, refer to the specific API documentation.

Clock Jitter Reduction

There are several ways to optimize the jitter of the TSN clock as mentioned below (ordered by the effects):

1. To reduce jitter with a GEI driver, set the interruption throttle interval as small as possible within the capability of the system. To set this interval, update the VIP parameter **TSN_CLK_INT_THROTTLE_INTERVAL**. The default value of this parameter is -1, which indicates that this feature is disabled. Setting it to a positive value will enable this feature. The unit of the value is in µs (microsecond). As per some tests, on a mainstream Intel board the value can be set to 1. What this means is that the interval between two interruptions is not larger than 1µs or **TSN_CLK_INT_THROTTLE_INTERVAL**.
2. In a multicore environment, it is possible to allocate a dedicated CPU core for the TSN clock interruption process. The API tsnClockIntReroute( ) is designed for this purpose. Use this API to reroute the interruption of one NIC to a specified CPU core.
3. Disable the power management feature by excluding the folder **FOLDER_PWR_MGMT** from the VIP.

3. SCHEDULED TRAFFIC WITH IEEE 802.1QBV

The IEEE 802.1Qbv standard guarantees timely delivery using a system-wide schedule shared among all bridges and endpoints in the system.

The schedule ensures that no two devices transmit to the same bridge or endpoint at the same time. The schedule defines a cycle time (for example, 500 microseconds) for the system-wide schedule.

For each communication flow in the system, the schedule also allots a timeslot (for example, 140- to 160-microseconds) within the cycle time when an endpoint can send network packets without interruption. This flow is identified on the local network by a media access control (MAC) address, a VLAN ID, and a Priority Code Point (PCP). A typical system defines multiple communication flows.

4. ABOUT VXWORKS TSN STREAM

The VxWorks time-sensitive networking (TSN) stream enables the VxWorks applications to send user datagram protocol (UDP), TCP, or layer 2 Ethernet packets on an IEEE 802.1 Qbv flow using the VxWorks network stack.

VxWorks performs a stream translation on the network packets of a TSN stream to add the flow media access control (MAC) address, VLAN ID, and priority code point (PCP) to the Ethernet packet header, and the packets are forwarded to the time-sensitive queue on the enhanced network driver (END) device.

TSN stream relies on the Precision Time Protocol (PTP) to keep in sync with the system-wide master clock, so make sure PTP is included. For more information, see the VxWorks 7 NTP and PTP Programmers Guide.

5. VXWORKS CONFIGURATION FOR TSN

Support for TSN IEEE 802.1Qbv is provided with several VxWorks source build (VSB) and VxWorks image project (VIP) options.

VSB Configuration

Option	Description
PTP	Enables Precision Time Protocol (PTP) support. For more information, see the VxWorks 7 NTP and PTP Programmer's Guide.
TSN	Enables Time-Sensitive Network (TSN) support.
GPTP	Enables Generalized Precision Time Protocol (gPTP) support. For more information, see the VxWorks 7 NTP and PTP Programmer's Guide.

VIP Configuration

Component	Description
INCLUDE_IPPTP	Provides support for the Precision Time Protocol module. For more information, see the VxWorks 7 NTP and PTP Programmer's Guide.
INCLUDE_IPPTP_SHOW	Includes support for PTP show routines.
INCLUDE_GPTP	Provides support for the Generalized Precision Time Protocol module. For more information, see the VxWorks 7 NTP and PTP Programmer's Guide.
INCLUDE_1588_CLOCK	Provides support for TSN clocks and timers.
INCLUDE_8021QBV	Includes support for TSN Stream (802.1Qbv).
INCLUDE_8021QBV_SHOW	Includes support for 802.1Qbv show routines.
INCLUDE_LEGACY_QBV_API	Legacy TSN support for IEEE 802.1Qbv (Legacy APIs) depending upon the configuration.

6. APIS FOR TIME-SENSITIVE NETWORKING

1. Clock and Timers

Function	Description
tsnClockIdGet( )	Get a TSN 1588 timer device ID.
tsnClockIntReroute( )	Reroute timer interrupts to a special CPU core.
tsnClockTimeGet( )	Get the current time of the TSN clock.
tsnTimerAllocate( )	Allocate a TSN 1588 timer.
tsnTimerRelease( )	Release a TSN 1588 timer.
tsnClockConnect( )	Connect a routine to the TSN 1588 clock cyclical (ticks) or one shot interrupt.
tsnClockDisable( )	Turn off the TSN 1588 clock cyclical (ticks) interrupts.
tsnClockEnable( )	Turn on the TSN 1588 clock cyclical or one shot interrupts.
tsnClockRateGet( )	Get the TSN 1588 cyclical (ticks) clock rate.
tsnClockRateSet( )	Set the TSN 1588 cyclical (ticks) clock rate.

2. Stream Setup

Function	Description
tsnStreamCreate( )	Create a TSN stream object.
tsnStreamDeviceSet( )	Bind the TSN stream to a TSN device.
tsnStreamDeviceGet( )	Get the bound TSN device of TSN stream.
tsnStreamCycleStartSet( )	Set the cycle start time of the TSN stream.
tsnStreamCycleStartGet( )	Get the cycle start time of the TSN stream.
tsnStreamPeriodSet( )	Set the period of the TSN stream.
tsnStreamPeriodGet( )	Get the period of the TSN stream.

Function	Description
tsnStreamOffsetSet( )	Set the time offset of the stream.
tsnStreamOffsetGet( )	Get the time offset of the stream.
tsnStreamVidGet( )	Get the vlan of the TSN stream.
tsnStreamVidSet( )	Set the VLAN ID of the TSN stream.
tsnStreamPcpSet( )	Set the priority of the TSN stream.
tsnStreamPcpGet( )	Get the priority of the TSN stream.
tsnStreamDstSet( )	Set the destination MAC of the TSN stream.
tsnStreamDstGet( )	Get the destination MAC of the TSN stream.
tsnStreamNameGet( )	Get the the ASCII string name of the TSN stream handle.
tsnStreamGet( )	Get the TSN stream handle based on the stream name.
tsnStreamDestroy( )	Delete a TSN stream object.
tsnStreamNextLaunchSetup( )	Set the next launch time of the TSN stream. When this function is called the next launch time is calculated, and the stream is sent only at that calculated launch time.

3. Stream IP Interception

Function	Description
tsnStreamIpDstSet( )	Set the destination IP and mask of the TSN stream.
tsnStreamIpSrcSet( )	Set the source IP and mask of the TSN stream.
tsnStreamIpPortSet( )	Set the destination port and source port of the TSN stream.
tsnStreamIpProtoSet( )	Set the TCP/UDP protocol of the TSN stream.
tsnStreamIpAccessEnable( )	Set the TCP/UDP protocol of the TSN stream.
tsnStreamIpAccessDisable( )	Disable the IP interception of the stream.

4. Stream Show

Function	Description
tsnStreamShow( )	Show the data base of the stream.
tsnStreamShowAll( )	Show the data base of all the streams.

7. TSNCONFIG - A JSON BASED TSN STREAM CONFIGURATION TOOL

1. TSN Stream Configuration Tool

tsnConfig is used for setting up TSN Streams and configuring hardware.

A gate control list (GCL) is a complicated structure and is not easy to configure in the traditional way by calling a function. tsnConfig, a generic TSN Stream configuration tool is provided to help ease the configuration.

tsnConfig is only used on the Tx side of a TSN stream. It reads a JSON-formated configuration file and does the TSN stream configuration per networking device.

An example JSON configuration file is below:

```
{
    "schedule": {                     // Schedule timing is mandatory
        "cycle_time": 1000000,        // Cycle time is 1000 usecs.
        "start_sec": 0,               // Schedule starts at { start_sec, start_nsec }
        "start_nsec": 0
    },
    "preemption": {                   // Preemption enabled (optional)
        "tclass_mask": 0x01           // Traffic class mask. TC 0 is preemptible,
                                      // other TCs are express.
    },
    "gate_control_list": [            // 802.1Qbv gate control list (optional).
                                      // Total gate time must be equal to cycle time.
        {
            "tclass_mask": 0x80,      // Traffic class mask. Only TC 7 can transmit.
            "time": 100000            // Gate time 100 usec, opens at 0 usec for
                                      // first gate.
        },
        {
            "tclass_mask": 0x40,      // Only TC 6 can transmit.
            "time": 200000            // Gate time 200 us, opens at 100 usec.
        },
        {
            "tclass_mask": 0x3f,      // TC 0-5 can transmit. Use for "best-effort"
                                      // traffic.
            "time": 700000            // Gate time 700 usec, opens at 300 usec.
        }
    ],
    "stream_objects": [               // At least one stream object is mandatory
        {
            "stream": {
                "name": "flow1",
                "dst_mac": "01:c0:ff:ee:00:04",
                "vid": 3000,
                "pcp": 7,             // normally pcp==tclass
                "tclass": 7,          // Stream maps to TC 7.
                "tx_time": {          // Transmission time based scheduling
                                      // (optional), usually no gate control list
                                      // should be defined
                    "offset": 50000   // Stream transmits at 50 usecs.
```

```
                }
            }
        },
        {
            "stream": {
                "name": "flow2",
                "dst_mac": "01:c0:ff:ee:00:05",
                "vid": 3001,
                "pcp": 6,
                "tclass": 6          // Stream maps to TC 6, transmitting in 100-300
                                     // usec window.
            },
            "ip": {                  // IP interception (optional)
                "dst_ip": "192.168.1.11",
                "dst_mask": "255.255.255.255",
                "dst_port": 0,       // available when proto == 6 or 17
                "src_ip": "192.168.1.10",
                "src_mask": "255.255.255.255",
                "src_port": 0,       // available when proto == 6 or 17
                "proto": 0           // IP protocol number. e.g. UDP: 17, TCP: 6
            }
        }
    ]
}
```

Note: Comments provided in the above JSON example file are only for explanatory purpose. The actual JSON format does not allow for comments.

tsnConfig provides two types of user interfaces:

- tsnConfig( ), a C API which can be called in a kernel application or C Shell. It is a program interface.

```
STATUS tsnConfig
 (
 const char *    pEthName,       /* Network device name */
 size_t          ethNameSize,    /* Network device name length */
                                 /* including '\0' */
 int             unit,           /* Network device unit number */
 TSN_CONF_OPT    opt,            /* Operation type */
 const char *    pConfFilePath   /* Configuration file path */
 );
```

- tsnconfig which is used in the command shell. It is a human-machine interaction interface that improves the user experience of manually running this tool.

```
-> cmd
[vxWorks *]# tsnconfig
tsnconfig <ethName> [options]:
      create <config file path> - Create TSN configuration for Ethernet interface
      flush - Clear all TSN configuration
      dump  - Dump all TSN configuration
      stats - Show TSN streams' statistics
      hwstats - Show TSN hardware statistics
```

To include tsnConfig( ) API and the tsnconfig shell command in the VIP, include components **INCLUDE_TSN_CONFIG** and **INCLUDE_TSN_CONFIG_CMD**.

Some examples of using tsnConfig both as an API and command shell interface are shown below:

- Read the TSN configuration from a configuration file. According to the configuration, create TSN streams and configure hardware for a NIC. Before applying the new configuration, sanity checks will be performed and the old TSN configuration will be removed.

```
-> tsnConfig("enetc", 6, 0, 0, "/romfs/tsn.json")
[vxWorks *]# tsnconfig enetc0 create "/romfs/tsn.json"
```

- Clear current TSN configuration for a NIC.

```
-> tsnConfig("enetc", 6, 0, 1)
[vxWorks *]# tsnconfig enetc0 flush
```

- Dump the current configuration of a NIC.

```
-> tsnConfig("enetc", 6, 0, 2)
[vxWorks *]# tsnconfig enetc0 dump
```

- Show statistics of TSN streams of a NIC.

```
-> tsnConfig("enetc", 6, 0, 3)
[vxWorks *]# tsnconfig enetc0 stats
```

- Show TSN hardware statistics of a NIC.

```
-> tsnConfig("enetc", 6, 0, 4)
[vxWorks *]# tsnconfig enetc0 hwstats
```

8. EXAMPLE TSN STREAM AND FRAME PRE-EMPTION CONFIGURATION ON NXP LS1028A-PA BOARD

1. TSN Stream (802.1Qbv) and TSN Clock

TSN Stream (802.1Qbv)

There are two ways to enable TSN Stream (802.1Qbv) on ENETC (networking chip on NXP LS1028A-PA board):

- Time Specific Departure (TSD)
- Time Gated - IEEE 802.1Qbv

Time Specific Departure (TSD)

A time specific departure capability enables the user to specify when a frame can be transmitted. When this capability is enabled, the device will delay the transmission of the frame so that it can be transmitted at the precisely specified time.

TSD is enabled on queue 7 which has the highest priority. Currently, TSN traffic will go through queue 7 and other traffic will go through queue 0.

Along with TSD, Multi-Queue and VLAN are also implemented. For Multi-Queue, there are 8 Tx queues(0~7) and 1 Rx queue (0). For VLAN, not all the VLAN features supported by ENETC are enabled. Only the features required by TSN Stream are enabled.

Mapping relationships among queues, priorities and traffic classes are as follows:

Queue No	Priority	Traffic Class (TC)
0	0	0
1	1	1
...	...	...
...	...	...
6	6	6
7	7	7

Limitations using TSD:

- The cycles and the time slots are maintained by the TSN Stream library. Due to software delays, it is possible that a packet might miss its time slot.
- Even though ENETC supports up to 16 Tx queues and 8 traffic classes, in order to guarantee the performance of TSD, only the highest priority queue(queue7 with TC7) is used. So multiple TSN streams share the same Tx queue.
- It is necessary to use ETF (Earliest TxTime First) to make sure that all packets from different streams (usually from different tasks) are in the order according to the Tx time in the Tx queue. For more information on how to configure ETF please refer to *VxWorks 7 Time-Sensitive Networking Programmer's Guide*.
- TSD only allows sending one packet from one stream, in its time slot of a cycle. With TSD, if you want to send multiple packets in one time slot, the solution is to bind multiple streams with one application.

WNDRVR

Time Gated - IEEE 802.1Qbv

A Time-Aware Shaper (TAS) as specified in IEEE 802.1 Qbv, also known as time gating, allows transmission from each traffic class to be scheduled relative to a known timescale. In order to achieve this, a transmission gate is associated with each traffic class and the state of transmission gate determines whether or not queued frames can be selected for transmission. The transmission gate can be in one of two states:

- Open: Queued frames are selected for transmission in accordance with the definition of the transmission selection algorithm associated with the traffic class.
- Closed: Queued frames are not selected for transmission.

Sample Gate Control List:

Entry	**0**	**1**	**2**	**3**	**4**
Intervals(ns)	100000	200000	100000	300000	300000
TC0	1(Open)	0(Close)	0(Close)	0(Close)	1(Open)
TC7	0(Close)	1(Open)	0(Close)	0(Close)	0(Close)
TC6	0(Close)	0(Close)	1(Open)	0(Close)	0(Close)
TC5	0(Close)	0(Close)	0(Close)	1(Open)	1(Open)

- A Gate Control List (GCL) associated with a port contains an ordered list of gate operations. Each gate operation specifies the gate states for all gates and a duration for these states.
- GCL contains five entries and each entry represents one gate. A gate defines how long it opens and for which traffic classes it opens. For example, in the table above, Gate0 opens for 100000ns and in this period, packets of Traffic Class0 can be transmitted. Gate4 opens for 300000ns and packets of Traffic Class0 and Traffic Class5 can be transmitted.
- A GCL is defined with a CycleTime.
- If there is no new GCL configured, the current GCL will repeat from the first gate in each cycle. So the sum of the gates time has to be less than or equal to the CycleTime.
- If a new GCL is configured, it will not take effect right away, but instead it will wait until the current cycle of the old GCL finishes.

Using Time Gated - IEEE 802.1Qbv, many of the limitations of TSD can be eliminated.

- The hardware controls the cycles and gates. This eliminates delays caused by software, thereby providing the best performance.
- 8 Tx queues and corresponding 8 Traffic Classes are fully utilized.
- Each stream can have its dedicated Tx queue, priority and Traffic Class. This eliminates the issue of having out of order packets.
- As long as the period of the Gate is long enough, there are no limitations in sending packets for one stream in one Gate.

Note: Developing a TSN Stream application with TGQbv is very similar to developing a normal socket or mux application. Please refer to the demo code at *installDir* **\vxworks-7\pkgs_v2\net\tsn\demo\demoTGQbv.c**

TSN Clock

There are two timers on NXP LS1028A-PA board and both of them support periodic timer interrupts and one-shot timer interrupts.

Both the starting time and periodic interval must be set in nanoseconds to an integer multiple of 1588 clock period (5ns). Interrupts can only be triggered at the start of a 1588 clock period. If the user configures the TSN clock for an interrupt rate which is not an integer multiple of 5ns the actual rate will be inaccurate. For example, setting a rate of 3 interrupts per second would actually trigger the interrupt every 333333330ns (the integer multiple of 5ns).

WNDRVR

Re-route is supported.

Throttling of the 1588 timer interrupt is not needed as it does not require coalescing. Unlike the i210, the 1588 timer does not share the interrupt with the ENETC device.

Note: There are three PPS (fixed interval period pulse generator) output but as of now they are not enabled.

2. Configuring Time-Sensitive Networking on ENETC

Before You Begin

These steps assume that you have created a VxWorks Source Build (VSB) project called **ls1028_vsb** based on the **nxp_layerscape_a72** BSP.

About This Task

These steps describe how to enable PTP, gPTP, TSN clock, jsnConfig and TSN Stream (802.1Qbv) on ENETC for the NXP LS1028A-PA board.

Procedure

1. Add the layers for TSN, PTP, gPTP to the VSB and rebuild the project.

```
cd ls1028_vsb
vxprj vsb config -s -add _WRS_CONFIG_PTP=y
vxprj vsb config -s -add _WRS_CONFIG_GPTP=y
vxprj vsb config -s -add _WRS_CONFIG_TSN=y
make -j [thread number]
```

2. Create a VxWorks Image Project (VIP) called **ls1028_vip** based on the VSB. For details, refer to the topic *Creating a VxWorks Kernel Image with Embedded DTB* in the *VxWorks 7 nxp_layerscape_a72 BSP Supplement Guide*.
3. Enable the LS1028A board support and add the ENETC END and 1588 drivers to the VIP.

```
cd ls1028_vip
vxprj component add INCLUDE_LS1028A_RDB_PA
vxprj component add INCLUDE_NXP_ENETC_VXB_END INCLUDE_ENETC_IEEE1588
```

4. Add components for PTP, gPTP and TSN stream to the VIP.

```
vxprj component add INCLUDE_IPPTP
vxprj component add INCLUDE_GPTP
vxprj component add INCLUDE_8021QBV INCLUDE_LEGACY_QBV_API INCLUDE_8021QBV_SHOW
```

5. Add component for the TSN clock to the VIP.

```
vxprj component add INCLUDE_1588_CLOCK
```

6. Add tsnConfig and tsnConfig command to the VIP.

WNDRVR

```
vxprj component INCLUDE_TSN_CONFIG INCLUDE_TSN_CONFIG_CMD
```

7. Add ROMFS for storing the configuration files of tsnConfig to the VIP.

```
vxprj component INCLUDE_ROMFS
mkdir romfs
```

8. Copy the TSN stream demo code to the VIP.

```
cp -r installDir\vxworks-7\pkgs_v2\net\tsn\demo .
```

9. Add the demo source files into the VIP project.

```
vxprj vip file add demo\*.c
```

10. Add additional utility components and build the VIP.

```
vxprj component add INCLUDE_PING INCLUDE_IPIFCONFIG_CMD INCLUDE_IFCONFIG INCLUDE_SHELL INCLUDE_
PCI_SHOW INCLUDE_IPATTACH INCLUDE_STANDALONE_DTB INCLUDE_STANDALONE_SYM_TBL
vxprj build
```

3. IEEE802.1Qbu Frame Pre-emption

Frame pre-emption is used in order to interrupt the transmission of long frames in favor of high-priority frames. As soon as the high-priority traffic is through, the remaining part of the interrupted frame can be transmitted.

This procedure allows for optimal bandwidth utilization of background traffic when used with 802.1Qbv Time-Aware Shaper (TAS) and enables low-latency communication in non-scheduled networks.

There are 2 MACs on ENETC (networking chip on NXP LS1028A-PA board):

- EMAC- Express MAC which is used for high-priority traffic.
- PMAC- Pre-emptable MAC which is used for low-priority traffic.

ENETC supports 8 traffic classes (TC0 ~ TC7 from low priority to high priority). By default, all the traffic classes are on the EMAC.

The diagram below shows the relationships between EMAC, PMAC and the MAC Merge Sublayer.

```
            +                           +
 High Priority Traffic         Low Priority Traffic
            |                           |
            |                           |
            v                           v
     +-------+---------+         +-------+--------+
     |                 |         |                |
     |                 |         |                |
     |      EMAC       |         |      PMAC      |
     |                 |         |                |
     |                 |         |                |
     +--------+--------+         +-------+--------+
              |                           |
```


WNDRVR

```
          |                          |
          v                          v
    +--------+-------------------------+--------+
    |             MAC Merge Sublayer            |
    |        (Frame Preemption happens here)    |
    +-------------------------------------------+
```

For more information on how the new hardware capability **IFCAP_QBU** can be used, refer to the demo code *installDir* **\vxworks-7\pkgs_v2\net\tsn\demo\demoQbu.c**

Two new options EIOCSQBU and EIOCGQBU are now available for muxIoctl( ) for setting and getting Qbu configuration. The data associated with them is an 8-bit vector (UINT8). Each bit of the vector stands for a traffic class.

Bit	Traffic Class (TC)
0	0
1	1
...	...
6	6
7	7

If the value of the bit is 1, it means the corresponding TC is on PMAC, otherwise it is on EMAC. Please refer to the demo code at *installDir* **\vxworks-7\pkgs_v2\net\tsn\demo\demoQbu.c** for an example of how the options are used.

The ENETC driver provides a test API called enetcStatsShow( ). This API takes one argument *unit* which is used to identify the ENETC to be used. The output of the API includes 2 counters:

- Rx preemption counter- Number of pre-empted frames received.
- Tx preemption counter- Number of pre-empted frames sent.

An example below:

```
->enetcStatsShow 0
enetc0:
Rx preemption counter:                 49
Tx preemption counter:                  0
```

4. Configuring Frame Pre-emption on ENETC

Before You Begin

These steps assume that you have created a VxWorks Source Build (VSB) project called **ls1028_vsb** based on the **nxp_layerscape_a72** BSP.

About This Task

These steps describe how to enable frame pre-emption (802.1Qbu) on ENETC for the NXP LS1028A-PA board.

WNDRVR

Note:

802.1Qbu has to work along with 802.1Qbv (TSN Stream). It is meaningless to use it alone.

Both Time Gated Qbv (TGQbv) and Time Specific Departure (TSD) can work with Qbu. For TSD this means only TC7 (TSN traffic) and TC0 (normal traffic) are really used. Other TCs are all idle. So the only meaningful usage of Qbu is to direct TC0 to PMAC. There is no such limitation for TGQbv. All 8 TCs (0~7) can be utilized and any of them can be routed to PMAC.

Procedure

1. Add the layers for TSN and PTP and rebuild the project.

```
cd ls1028_vsb
vxprj vsb config -s -add _WRS_CONFIG_PTP=y
vxprj vsb config -s -add _WRS_CONFIG_TSN=y
make -j [thread number]
```

2. Create a VxWorks Image Project (VIP) called **ls1028_vip** based on the VSB. For details, refer to the topic *Creating a VxWorks Kernel Image with Embedded DTB* in the *VxWorks 7 nxp_layerscape_a72 BSP Supplement Guide*.
3. Enable the LS1028A board support and add the ENETC END and 1588 drivers to the VIP.

```
cd ls1028_vip
vxprj component add INCLUDE_LS1028A_RDB_PA
vxprj component add INCLUDE_NXP_ENETC_VXB_END INCLUDE_ENETC_IEEE1588
```

4. Add components for PTP and TSN stream to the VIP.

```
vxprj component add INCLUDE_IPPTP
vxprj component add INCLUDE_8021QBV INCLUDE_LEGACY_QBV_API INCLUDE_8021QBV_SHOW
```

5. Copy the TSN stream demo code to the VIP.

```
cp -r installDir\vxworks-7\pkgs_v2\net\tsn\demo .
```

6. Add the demo source files into the VIP project.

```
vxprj vip file add demo\*.c
```

7. Add additional utility components and build the VIP.

```
vxprj component add INCLUDE_PING INCLUDE_IPIFCONFIG_CMD INCLUDE_IFCONFIG INCLUDE_SHELL INCLUDE_
PCI_SHOW INCLUDE_IPATTACH INCLUDE_STANDALONE_DTB INCLUDE_STANDALONE_SYM_TBL
vxprj build
```

8. To test this feature, you can run background traffic using **iperf**.

   Download iperf:

```
cd installDir\pkgs_v2\net\
mkdir app
cd app
git clone https://github.com/Wind-River/vxworks7-layer-for-iperf.git iperf3
```


WNDRVR

Add iperf to VSB and rebuild the VSB

```
vxprj vsb config -s -add _WRS_CONFIG_IPERF3=y
make -j [thread number]
```

Add iperf to VIP and rebuild the VIP

```
vxprj component add INCLUDE_IPERF3
vxprj build
```

9. On the sender side, run the following commands:

```
->ptpdAdd "enetc0",2,1,1,0,128,-3,1
->ptpdStart
->enetcStatsShow 0
->demoQbuSet "enetc", 0, 1
->demoStreamCommonInit "flow1002", "enetc",0, 0,1000000, 200000, 3001,5, "a0:36:9f:57:31:c8"
->demoStreamMuxTx "flow1002"
->sp iperf3,"-c 192.168.0.2 -u -t 20 -p 10000 -i 20 -b 0 -f m -l 1450"
->demoStreamMuxTxStop
->enetcStatsShow 0
```

10. On the receiver side, run the following commands:

```
->ptpdAdd "enetc0",2,0,1,0,128,-3,1
->ptpdStart
->sp iperf3,"-s -p 10000 -B 0.0.0.0"
->enetcStatsShow 0
->demoQbuSet "enetc", 0, 1
->demoStreamCommonInit "flow1002", "enetc",0, 0,1000000, 200000, 3001,5, "a0:36:9f:57:31:c8"
->demoStreamMuxRx "flow1002"
->enetcStatsShow 0
```


WNDRVR

9. SENDING AND RECEIVING OVER A TSN STREAM

1. TSN Stream Setup

A VxWorks time-sensitive networking (TSN) stream is the representation of one IEEE 802.1Qbv flow.

A new TSN stream needs to be created for each outgoing flow. The initial setup of a TSN stream involves the following:

- Creating a named TSN_STREAM object
- Binding the TSN_STREAM object to an end device
- Setting the IEEE 802.1Qbv schedule parameters on the TSN_STREAM object

Each of these steps is shown in the following examples.

Create a Named Stream Handle

```
TSN_STREAM streamHandle;
streamHandle = tsnStreamCreate("flow1003");
```

Bind the Stream to an END Device

```
     STATUS status;
     /* Bind the stream handle to gei1. */
status = tsnStreamDeviceSet( streamHandle, "gei", 1);
```

Set the Various IEEE 802.1Qbv Schedule Parameters for the Stream

```
UINT8 flowMac1003[6] = {0x03, 0x00, 0x5e, 0xa0, 0x03, 0xe9};

/* Start the stream right away */
struct timespec ts;
ts.tv_sec=0;
ts.tv_nsec=0;
status = tsnStreamCycleStartSet( streamHandle, &ts );

/* Set the schedules time cycle period to 1000000 nanoseconds. */
status= tsnStreamPeriodSet( streamHandle, 1000000L );

/* Set up the stream to start sending at 260000 nanoseconds.
status = tsnStreamOffsetSet( streamHandle, 260000L );

/* Use VLAN ID 3000 */
status = tsnStreamVidSet(streamHandle, 3000 );

/* Set the stream port priority to 5. */
status = tsnStreamPcpSet( streamHandle, 5 );

/* Set the stream MAC address to 03:00:5e:a0:03:e9. */
status= tsnStreamDstSet( streamHandle, flowMac1003);
```

The example file **demoStreamCommon.c** under *installDir* **vxworks-7/pkgs_v2/net/tsn/demo** illustrates how to set up an initial TSN stream.

2. Sending and Receiving

After the initial setup of a time-sensitive networking (TSN) stream, there are three ways to send and receive network packets over the stream.

- Socket binding – create a universal datagram protocol (UDP) or TCP socket and bind it to a TSN stream.
- IP interception – filter all UDP and TCP sockets through a set of IP parameters configured into each TSN stream.
- L2 Ethernet payloads – send the payload for L2 Ethernet packets using a TSN stream.

3. TSN Stream Socket Binding

In the time-sensitive networking (TSN) stream socket binding, a single TCP or user datagram protocol (UDP) socket is bound to a TSN stream.

All packets being sent out of a bound socket go through TSN stream translation, this is a static and fixed way with higher performance than using **IP Interception**. The binding stays in effect until the socket is closed. To set up socket binding for the sender, set up a TSN stream, and create a UDP or TCP socket using the sockLib( ) application program interfaces (APIs). Bind the socket to the TSN stream with the following call:

```
setsockopt(socketFd, SOL_SOCKET, SO_X_QBV, (void*)streamName, TSN_STREAMNAMSIZ)
```

All the packets transmitted from that socket are sent using the TSN stream's schedule parameters. The packet automatically includes the proper packet headers for TSN.

Socket binding is not used on the receiving end of a TSN stream. To receive UDP or TCP packets from a TSN flow, use the sockLib( ) APIs to receive UDP or TCP packets. You will also need to set up the enhanced network driver (END) to accept packets from the flow MAC address.

Note: If the TSN stream uses a VLAN ID, make sure that VLAN is created.

Socket Binding Example on page 21 How to set up a TSN stream, set up a UDP soqAcket, bind the socket to the TSN stream and then send and receive over the TSN stream.

3.1. Socket Binding Example

How to set up a TSN stream, set up a UDP soqAcket, bind the socket to the TSN stream and then send and receive over the TSN stream.

About This Task

The files **demoStreamSocketTx.c**, **demoStreamSocketRx.c**, and **demoStreamCommon.c** under **vxworks-7/net/ipnet/tsn/stream/demo** show how to set up a TSN stream, set up a UDP **soqAcket**, bind the socket to the TSN stream and then send and receive over the TSN stream.

Before You Begin

This example uses the following parameters:

- TSN flow assigned to 1-millisecond cycle time
- Transmit window starts at 200 microseconds
- VLAN 3000
- Priority code point (PCP) of 5
- A flow identified by MAC address 03:00:5e:a0:03:e9
- END device name is gei0
- Receiver IP address is 10.1.1.10

Procedure

1. On the receiver, enter the following commands:

```
-> demoStreamCommonInit "flow1001", "gei", 0, 0, 1000000, 200000, 3000, 5, "03:00:5e:a0:03:e9"
-> demoStreamSocketRx "flow1001", 5000
```

2. On the sender, enter the following commands:

```
-> demoStreamCommonInit "flow1001", "gei", 0, 0, 1000000, 200000, 3000, 5, "03:00:5e:a0:03:e9"
-> demoStreamSocketTx "flow1001", "10.1.1.10", 5000, 0
```

3. To stop the demo on the receiver, enter the following commands:

```
-> demoStreamSocketRxStop
```

4. To stop the demo on the sender, enter the following commands:

```
-> demoStreamSocketTxStop
```

Parent topic: TSN Stream Socket Binding on page 21

4. IP Interception

IP Interception enables stream translation on all network traffic that satisfies a set of filters for a time-sensitive networking (TSN) stream.

IP Interception is dynamic and changeable, and provides more flexibility than TSN stream socket binding. IP interception can be enabled, disabled and updated with new filters while the socket is in running state. Self-contained applications that set up their own sockets to send and receive data can remain unchanged when IP interception is set up before the application starts.

IP interception is not used on the receiving end of a TSN stream. To receive user datagram protocol (UDP) or TCP packets from a TSN flow, use the sockLib( ) APIs to receive UDP or TCP packets. If the TSN stream uses a VLAN ID, make sure that VLAN is created. You will also need to set up the enhanced network driver (END) driver to accept packets from the flow media access control (MAC) address.

How to Use IP Interception

To set up IP interception:

- Set up a TSN stream.
- Set up filtering for source and destination IP addresses, the port, and protocol (UDP or TCP).
- Set up filtering using the functions tsnStreamIpDstSet( ), tsnStreamIpSrcSet( ), tsnStreamIpPortSet( ), and tsnStreamIpProtoSet( ).
- After the filter is set, call tsnStreamIpAccessEnable( ) to start IP interception

All socket send calls are intercepted, and those packets that match the filter in an enabled TSN stream are rerouted to TSN stream translation. Packets that fail the pattern matches enabled TSN streams are sent to the enhanced network driver (END) device send queue with no change.

This setup ensures all socket send calls are intercepted, and those packets that match the filter in an enabled TSN stream are rerouted to TSN stream translation.

To stop the IP interception process for a stream use tsnStreamIpAccessDisable( ).

IP Interception Filters

When a TSN stream is first created, all filters are set to wildcard values by default. If filter values are left as default values when tsnStreamIpAccessEnable( ) is called, then all packets sent out using the send( ) sockLib( ) API go through the TSN stream translation.

Filtering is set up using the functions tsnStreamIpDstSet( ), tsnStreamIpSrcSet( ), tsnStreamIpPortSet( ), and tsnStreamIpProtoSet( ). After a filter is set, call tsnStreamIpAccessEnable( ) to start IP interception.

All socket send calls are intercepted, and those packets that match the filter in a TSN stream are rerouted to TSN stream translation. Packets that fail the pattern matches on filters of all enabled TSN streams are sent to the END device send queue with no change. Use tsnStreamIpAccessDisable( ) to stop the IP interception process for a stream.

Filtering Using an IP Address Mask

Source and destination IP addresses can be filtered using an IP address mask. The following example matches all source IP addresses between 192.168.0.0 and 192.168.0.255.

```
-> tsnStreamIpSrcSet (stream, inet_addr("192.168.0.0"), inet_addr(255.255.255.0))
```

Filtering on an IP Port

When filtering on the IP port, a value of zero is a wildcard match. The following example filters packets sent to destination port 5000 and from any source port:

```
-> tsnStreamIpPortSet(stream, htons (5000), htons (0))
```

tsnStreamIpProtoSet( ) sets the TCP/UDP (an other) protocols of the TSN stream for IP interception. The initial value is zero, meaning all protocols will match.

Running the IP Interception Demo on page 24 How to set up a time-sensitive networking (TSN) stream, the IP interception filters and enable IP interception.

4.1. Running the IP Interception Demo

How to set up a time-sensitive networking (TSN) stream, the IP interception filters and enable IP interception.

About This Task

The files **demoStreamSocketTx.c**, **demoStreamSocketRx.c**, and **demoStreamCommon.c** under **vxworks-7/net/ipnet/tsn/stream/demo** show how to set up a TSN stream, the IP interception filters and enable IP interception.

Before You Begin

This example uses the following parameters:

- TSN flow assigned to 1-millisecond cycle time
- Transmit window starts at 200 microseconds
- VLAN 3000
- Priority code point (PCP) of 5
- Flow is identified by MAC address 03:00:5e:a0:03:e9
- The END device name is gei0
- The sender IP address is 128.224.158.74
- The receiver IP address is 128.224.159.6

Procedure

1. On the receiver, enter the following commands:

```
-> demoStreamCommonInit "flow1003", "gei", 0, 0, 1000000, 200000, 3000, 5, "03:00:5e:a0:03:e9"
-> demoStreamSocketRx "flow1003", 5000
```

2. On the sender, enter the following commands:

```
-> demoStreamCommonInit "flow1003", "gei",0, 0,1000000, 200000, 3000, 5, "03:00:5e:a0:03:e9"
-> demoStreamIpAccessSet "flow1003",0,"128.224.158.76",5000,"128.224.159.6",0
-> demoStreamSocketTx "flow1003","128.224.158.76",5000,1
```

   The sender sends three messages per second to the receiver. On the receiver, you should see network statistics printed out three times per second.

3. Stop the demo on the receiver by entering the following:

```
-> demoStreamSocketRxStop
```

4. Stop the demo on the sender by entering the following:

```
-> demoStreamSocketTxStop
```

Parent topic: IP Interception on page 22

5. L2 Ethernet Payloads

L2 Ethernet payloads for time-sensitive networking (TSN) streams perform TSN stream translation on Open Systems Interconnection (OSI) layer 2 Ethernet packets. The caller specifies the Ethernet payload to be sent, and the TSN stream translation adds the Ethernet frame headers for the TSN stream.

To send an L2 Ethernet payload on a TSN stream, first set up the TSN stream, then set up an mBlk structure from your enhanced network driver (END) device by calling endPoolTupleGet( ) and setting up all the structure parameters for your message. As part of the setup, set **mBlkPktHdr.streamCookie** to point to your TSN stream handle. Pass the mBlk structure to muxSend( ).

TSN streams are not needed on the receiving end of a TSN flow. To receive raw Ethernet packets from a TSN flow, use the muxLib APIs to receive Ethernet packets. You only need to set up the END driver to accept packets from the flow media access control (MAC) address.

Note: If the TSN stream uses a VLAN ID, make sure that VLAN is created.

L2 Ethernet Payload Example on page 25 How to set up a TSN stream and use the muxLib( ) APIs to send and receive messages on the TSN flow.

5.1. L2 Ethernet Payload Example

How to set up a TSN stream and use the muxLib( ) APIs to send and receive messages on the TSN flow.

About This Task

The files **demoStreamMuxTx.c**, **demoStreamMuxRx.c**, and **demoStreamCommon.c** located under **vxworks-7/net/ipnet/tsn/stream/demo** show how to set up a TSN stream and use the muxLib( ) APIs to send and receive messages on the TSN flow.

Before You Begin

This example uses the following parameters:

- TSN flow assigned to 1-millisecond cycle time
- Transmit window starts at 200 microseconds
- VLAN 3000
- Priority code point (PCP) of 5
- Flow identified by MAC address 03:00:5e:a0:03:e9
- The END device name is gei0

Procedure

1. On the receiver, enter the following commands:

```
-> demoStreamCommonInit "flow1001", "gei",0, 0,1000000, 200000, 3000,5, "03:00:5e:a0:03:e9"
-> demoStreamMuxRx "flow1001"
```

2. On the sender, enter the following commands:

```
-> demoStreamCommonInit "flow1001", "gei",0, 0,1000000, 200000, 3000,5, "03:00:5e:a0:03:e9"
-> demoStreamMuxTx "flow1001"
```

The sender sends out three packets per second to the receiver. On the receiver, you should see timing statistics printed out three times per second.

Parent topic: L2 Ethernet Payloads on page 25

6. Using the ETF Queuing Discipline to Transmit Packets

Use the **ETF** (**Earliest TxTime first**) queuing discipline (**qdisc**) feature in your **TSN** application to ensure that packets arrive in the device transmission queue in the order they should be sent based on the tagged **TxTime**.

ETF queuing ensures packets are buffered in the network stack interface queue and transmitted later according to their launch time (**TxTime**) minus a configurable delta factor.

Use ETF to handle multiple tasks that send packets tagged with a TxTime at different offsets into the cycle. For example, If two different tasks are sending packets through different TSN streams where:

- The cycle is 1000 microseconds
- Task 1 sends the packets via stream 1 with a 100 microsecond offset
- Task 2 sends the packets via stream 2 with a 200 microsecond offset

If task 1 sends two packets through stream 1 before task 2 sends one packet through stream 2, an incorrect launch-time order is put in the network interface transmit queue: 1100, 2100, 1200 microseconds. To resolve this issue, use the ETF queuing discipline where the proper launch-time order is set: 1100, 1200, 2100 microseconds.

Creating an ETF Enabled Project from the Command Line on page 26 You must create a VxWorks image project (VIP) and associate it with a VSB project that includes Time-Sensitive Networking (TSN) options.
Creating ETF Enabled VSB and VIP Projects in Workbench on page 27 You must create a VxWorks image project (VIP) and associate it with a VSB project that includes Time-Sensitive Networking (TSN) and ETF options.
Setting up the Network Interface for ETF on page 29 After you configure your VxWorks projects for ETF, you must enable the functionality on the network interface.
Testing your ETF Queue Setup on page 30 You can test your ETF setup by using the TSN demo.

6.1. Creating an ETF Enabled Project from the Command Line

You must create a VxWorks image project (VIP) and associate it with a VSB project that includes Time-Sensitive Networking (TSN) options.

Before You Begin

To create, configure, and build a VSB from the command line, you must set up the wrtool utility appropriately by pointing it to the right workspace. For more information, see the *VxWorks 7 Configuration and Build Guide: Creating a VxWorks Source Build Project from the Command Line*.

Procedure

1. Create the VSB project.

```
$ vxprj cmd vxprj vsb create -force -S -bsp itl_generic -cpu CORE -lp64 vsb_stream
$ cd vsb_stream/
```

2. Configure the VSB to use PTP by enabling: PTP, TSN, IPNET_QOS.

```
$ vxprj vsb config -w  -add _WRS_CONFIG_PTP=y -add _WRS_CONFIG_TSN=y -add _WRS_CONFIG_IPNET_QOS
=y make -j [thread number]
```

3. Create a VIP based on the VSB you created in step 1.

```
$vxprj create -force itl_generic llvm -vsb vsb_stream vip_stream
$cd vip_stream/
```

4. Add components to enable PTP and TSN streaming.

```
$vxprj component add INCLUDE_IPPTP
$vxprj component add INCLUDE_8021QBV INCLUDE_LEGACY_QBV_API INCLUDE_8021QBV_SHOW
```

5. Copy the TSN Stream Demo code to the VIP.

```
$cp -r ${WIND_BASE}/pkgs_v2/net/tsn/demo ./
```

6. Add the demo source code into the VIP project.

```
$vxprj vip file add demo/*.c
```

7. Add components from the IPNET_QOS layer.

```
$vxprj component add INCLUDE_IPQUEUE_CONFIG_CMD INCLUDE_IPQOS_CMD
```

8. Add the Utility components.

```
$vxprj component add INCLUDE_PING  INCLUDE_IPIFCONFIG_CMD INCLUDE_IFCONFIG INCLUDE_SHELL INCLUD
E_PCI_SHOW INCLUDE_IPATTACH INCLUDE_STANDALONE_DTB INCLUDE_STANDALONE_SYM_TBL
```

9. Build the project.

```
$vxprj build
```

Parent topic: Using the ETF Queuing Discipline to Transmit Packets on page 26

6.2. Creating ETF Enabled VSB and VIP Projects in Workbench

You must create a VxWorks image project (VIP) and associate it with a VSB project that includes Time-Sensitive Networking (TSN) and ETF options.

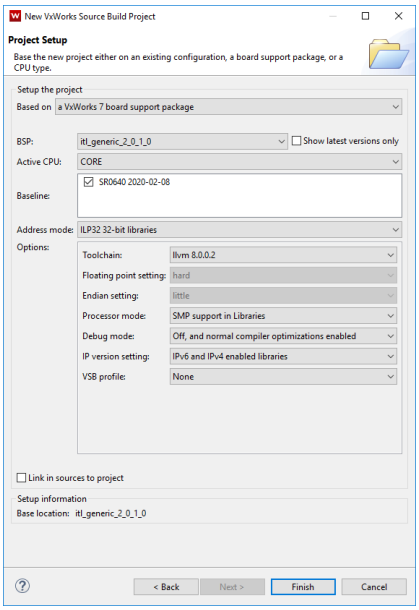
Before You Begin

You should know how to create, configure, and build a VSB project. For more information, see the *VxWorks 7 Configuration and Build Guide: Creating a VxWorks Source Build Project Using Workbench*.

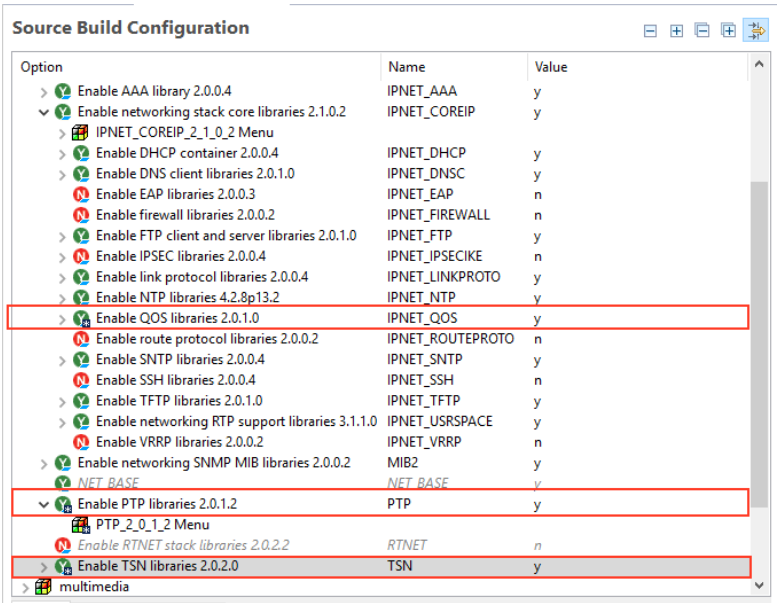
Procedure

1. Create a VxWorks Source Build project. In the BSP box, select itl_generic_2_0_1_0 and in the Active CPU box, select CORE.

WNDRVR

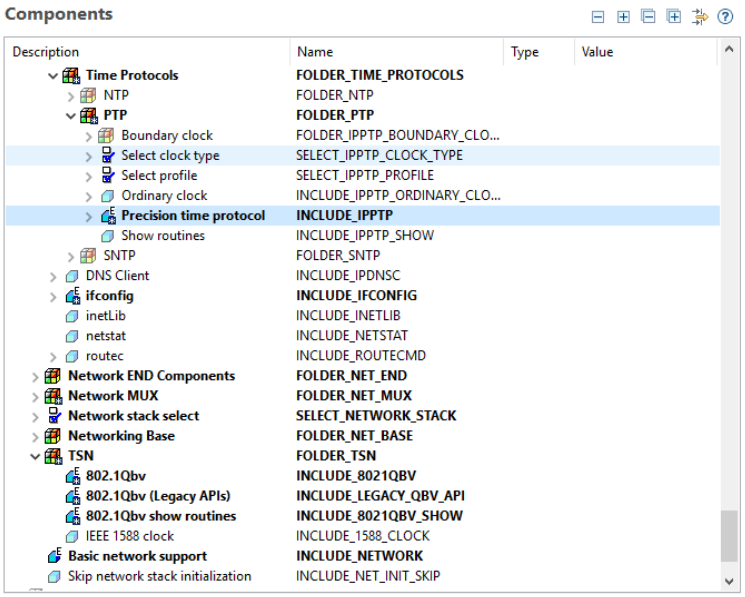


2. Expand the VSB project in the Project Explorer and double-click Source Build Configuration and enable these components:
   - **IPNET_QOS**
   - **PTP**
   - **TSN**



3. Build the VSB project.
4. Create a VxWorks Image Project and base it on the source build project you just created. For more information, see the *VxWorks 7 Configuration and Build Guide: Creating a VxWorks Kernel Image Project Using Workbench*.
5. Expand the VIP project in the Project Explorer and double-click Kernel Configuration and enable these options:
   - **INCLUDE_IPPTP**
   - **INCLUDE_8021QBV**
   - **INCLUDE_LEGACY_QBV_API**
   - **INCLUDE_8021QBV_SHOW**
   - **INCLUDE_PING**

- **INCLUDE_IPIFCONFIG_CMD**
- **INCLUDE_IFCONFIG**
- **INCLUDE_SHELL**
- **INCLUDE_PCI_SHOW**
- **INCLUDE_IPATTACH**
- **INCLUDE_STANDALONE_DTB**
- **INCLUDE_STANDALONE_SYM_TBL**



6. Copy **${WIND_BASE}/pkgs_v2/net/tsn/demo** to the VIP directory.
7. Refresh and build the VIP project.

Parent topic: Using the ETF Queuing Discipline to Transmit Packets on page 26

6.3. Setting up the Network Interface for ETF

After you configure your VxWorks projects for ETF, you must enable the functionality on the network interface.

Procedure

1. Add a **Multiband Container** (**MBC**) queue to be the root of a device.

```
[vxWorks *]#qc queue add dev [device] mbc bands 2 default_band 1
```

   The MBC queue has two bands including a default band.

2. Attach an ETF queue with a delta time to the first band of the MBC queue created in step 1.

```
[vxWorks *]#qc queue add dev [device] parent 1 handle [queue id] etf delta [latency nsec]
```

   - The ETF queue is a leaf queue under a MBC queue and it should not be in the default band.

- The delta time is an estimated system latency in nanoseconds from the ETF queue to the MAC. It is recommended you set the delta to100000 ns. It also should be less than the maximum of the valid launch time range for the MAC. For example, on an I210 NIC, the launch time is valid in [systime-0.5s] - [systime+0.5s]. Therefore, the delta should be less than 0.5s.

3. Add a filter to the MBC to ensure that all packets with a TxTime are sent to the ETF queue specified in step 2.

```
[vxWorks *]#qc filter add dev [device] parent 1 txtime flowid [queue id]
```

The ETF filter ensures the packets are based on the TxTime. All other packets go to the default band of the MBC which is usually a FIFO queue.

Parent topic: Using the ETF Queuing Discipline to Transmit Packets on page 26

6.4. Testing your ETF Queue Setup

You can test your ETF setup by using the TSN demo.

About This Task

In this example:

- There are two streams used to test ETF discipline where the stream 1 offset must be greater than the stream 2 offset.
- The sequence number of stream 1 starts at 0 and the sequence number of stream 2 starts at 1000.
- This test only allows for sending eight packets each time.
- The IP address in the command lines below is the receiving device IP address.
- The MAC address in the command lines below is the receiving device MAC address.
- The cycle should be 1 millisecond or longer.

Before You Begin

Create an ETF enabled project as described in Creating an ETF Enabled Project from the Command Line on page 26 or Creating ETF Enabled VSB and VIP Projects in Workbench on page 27 which include a step to copy the TSN demo to your project.

Procedure

1. Add a **Multiband Container** (**MBC**) queue as root to the **gei1** device where the MBC queue has two bands including a default band.

```
[vxworks *]#qc queue add dev gei1 mbc bands 2 default_band 1
```

2. Attach an ETF queue with a delta of 100000 nanoseconds to the first band of the MBC you added in step 1.

```
[vxworks *]#qc queue add dev gei1 parent 1 handle 1000 etf delta 100000
```

3. Add a filter to the MBC so that all packets with TxTime are sent to the ETF queue created in step 2.

```
[vxworks *]#qc filter add dev gei1 parent 1 txtime flowid 1000
```

4. Start the **PTP** master and slave on separate devices.
5. For the transmitting device, initialize two streams where stream 1 offset is longer than the stream 2 offset.

```
demoStreamCommonInit "flow1006", 8, "gei",1, 0,1000000, 900000, 3006,5, "a0:36:9f:97:9f:fd
"
demoStreamCommonInit "flow1007", 8, "gei",1, 0,1000000, 100000, 3007,5, "a0:36:9f:97:9f:fd"
```

6. Create sockets and send streams.

```
demoEtfStreamSocketTx "flow1006",8,"flow1007",8,"192.168.0.1",5000,0
```

7. Stop the transmitting task.

```
demoEtfStreamSocketTxStop
```

8. For the receiving device, initialize two streams where stream 1 offset is longer than the stream 2 offset.

```
demoStreamCommonInit "flow1006", 8,  "gei",1, 0,1000000, 900000, 3006,5, "a0:36:9f:97:9f:fd"
demoStreamCommonInit "flow1007", 8, "gei",1, 0,1000000, 100000, 3007,5, "a0:36:9f:97:9f:fd"
```

9. Create socket and receive streams.

```
demoEtfStreamSocketRx "flow1006","flow1007","192.168.0.1",5000
```

Results Without ETF enabled

On the receiving device, the packets are received in their transmitted order in the ETF task and not in TxtTme order of transmitting device. For example:

```
sequence: 17 ;Tx TIME 731:427900000; Rx TIME 731:427907868; Tx-Rx cost: 7868 ns; Tx-Rx Jitter:932
ns !
sequence: 18 ;Tx TIME 731:428900000; Rx TIME 731:428906982; Tx-Rx cost: 6982 ns; Tx-Rx Jitter:-88
6 ns !
sequence: 19 ;Tx TIME 731:429900000; Rx TIME 731:429906903; Tx-Rx cost: 6903 ns; Tx-Rx Jitter:-79
ns !
sequence: 20 ;Tx TIME 731:430900000; Rx TIME 731:430907001; Tx-Rx cost: 7001 ns; Tx-Rx Jitter:98 n
s !
sequence: 1017 ;Tx TIME 731:430100000; Rx TIME 731:430911561; Tx-Rx cost: 811561 ns; Tx-Rx Jitter:
-13615 ns !
sequence: 1018 ;Tx TIME 731:430100000; Rx TIME 731:430916121; Tx-Rx cost: 816121 ns; Tx-Rx Jitter:
4560 ns !
sequence: 1019 ;Tx TIME 731:430100000; Rx TIME 731:430920681; Tx-Rx cost: 820681 ns; Tx-Rx Jitter:
4560 ns !
sequence: 1020 ;Tx TIME 731:430100000; Rx TIME 731:430925241; Tx-Rx cost: 825241 ns; Tx-Rx Jitter:
4560 ns !
sequence: 21 ;Tx TIME 732:427900000; Rx TIME 732:427907964; Tx-Rx cost: 7964 ns; Tx-Rx Jitter:963
ns !
sequence: 22 ;Tx TIME 732:428900000; Rx TIME 732:428907069; Tx-Rx cost: 7069 ns; Tx-Rx Jitter:-89
5 ns !
sequence: 23 ;Tx TIME 732:429900000; Rx TIME 732:429906999; Tx-Rx cost: 6999 ns; Tx-Rx Jitter:-70
ns !
sequence: 24 ;Tx TIME 732:430900000; Rx TIME 732:430906904; Tx-Rx cost: 6904 ns; Tx-Rx Jitter:-95
```

```
ns !
sequence: 1021 ;Tx TIME 732:430100000; Rx TIME 732:430911464; Tx-Rx cost: 811464 ns; Tx-Rx Jitter:
-13777 ns !
sequence: 1022 ;Tx TIME 732:430100000; Rx TIME 732:430916024; Tx-Rx cost: 816024 ns; Tx-Rx Jitter:
4560 ns !
sequence: 1023 ;Tx TIME 732:430100000; Rx TIME 732:430920584; Tx-Rx cost: 820584 ns; Tx-Rx Jitter:
4560 ns !
sequence: 1024 ;Tx TIME 732:430100000; Rx TIME 732:430925144; Tx-Rx cost: 825144 ns; Tx-Rx Jitter:
4560 ns !
```

Results With ETF enabled

When ETF and PTPD tasks are enabled, the packets are received by the receiving device in the TxTtime order from the transmitting device. For example:

```
sequence: 1017 ;Tx TIME 64:84100000; Rx TIME 64:84100861; Tx-Rx cost: 861 ns; Tx-Rx Jitter:-4 ns !
sequence: 17 ;Tx TIME 64:84900000; Rx TIME 64:84900862; Tx-Rx cost: 862 ns; Tx-Rx Jitter:-6164 ns
!
sequence: 1018 ;Tx TIME 64:85100000; Rx TIME 64:85100863; Tx-Rx cost: 863 ns; Tx-Rx Jitter:2 ns !
sequence: 18 ;Tx TIME 64:85900000; Rx TIME 64:85900864; Tx-Rx cost: 864 ns; Tx-Rx Jitter:2 ns !
sequence: 1019 ;Tx TIME 64:86100000; Rx TIME 64:86100880; Tx-Rx cost: 880 ns; Tx-Rx Jitter:17 ns !
sequence: 19 ;Tx TIME 64:86900000; Rx TIME 64:86900865; Tx-Rx cost: 865 ns; Tx-Rx Jitter:1 ns !
sequence: 1020 ;Tx TIME 64:87100000; Rx TIME 64:87100866; Tx-Rx cost: 866 ns; Tx-Rx Jitter:-14 ns
!
sequence: 20 ;Tx TIME 64:87900000; Rx TIME 64:87907219; Tx-Rx cost: 7219 ns; Tx-Rx Jitter:6354 ns
!
sequence: 1021 ;Tx TIME 65:84100000; Rx TIME 65:84100856; Tx-Rx cost: 856 ns; Tx-Rx Jitter:-10 ns
!
sequence: 21 ;Tx TIME 65:84900000; Rx TIME 65:84900857; Tx-Rx cost: 857 ns; Tx-Rx Jitter:-6362 ns
!
sequence: 1022 ;Tx TIME 65:85100000; Rx TIME 65:85100857; Tx-Rx cost: 857 ns; Tx-Rx Jitter:1 ns !
sequence: 22 ;Tx TIME 65:85900000; Rx TIME 65:85900858; Tx-Rx cost: 858 ns; Tx-Rx Jitter:1 ns !
sequence: 1023 ;Tx TIME 65:86100000; Rx TIME 65:86100867; Tx-Rx cost: 867 ns; Tx-Rx Jitter:10 ns !
sequence: 23 ;Tx TIME 65:86900000; Rx TIME 65:86900852; Tx-Rx cost: 852 ns; Tx-Rx Jitter:-6 ns !
sequence: 1024 ;Tx TIME 65:87100000; Rx TIME 65:87100852; Tx-Rx cost: 852 ns; Tx-Rx Jitter:-15 ns
!
sequence: 24 ;Tx TIME 65:87900000; Rx TIME 65:87906933; Tx-Rx cost: 6933 ns; Tx-Rx Jitter:6081 ns
!
```

Note:

In the demo task, packets 17 and 1017 are in the same cycle. Normally, sequence 17 (stream 1) is always after sequence 1017 (stream 2) with ETF.

Parent topic: Using the ETF Queuing Discipline to Transmit Packets on page 26

10. TSN CLOCK DEMO

Use the code in **tsnclock_dkm.c** to synchronize the clocks on two TSN devices.

About This Task

The demo causes the Software-Defined Pin 0 (SDP0) on the Intel I210 network controller to toggle its ouput according to the 1588 timer.

Before You Begin

Use two devices that have an Intel I210 controller.

Configure VxWorks for TSN clock and PTP. Omit **INCLUDE_IPPTP_ORDINARY_CLOCK** and **INCLUDE_IPPTP_BOUNDARY_CLOCK**_*x*. These components automatically start the PTP daemon in a default mode. Support for Qav and Qbv is not required for this demo. For more information, see VxWorks Configuration for TSN.

About This Task

This procedure assumes that the device name for the Intel I210 NIC is **gei1** on both targets. The Intel I210 controllers of the two target systems must be directly connected to each other with no switches in between.

Procedure

1. Add **tsnclock_dkm.c** to your VIP project, and build the project.

   Alternatively, you can use the file in a DKM project, and download the object to the target system.

   The **tsnclock_dkm.c** is located in the **tsn/demo** directory of your installation.

2. Boot two target systems with your VxWorks image for your network nodes.
3. On the PTP master node, bind the PTP daemon to Ethernet interface gei1 and start it.

   Bind the PTP daemon as a master clock mode, with clock quality 128 using IEEE802.1AS PTP protocol. For more information, see the **ipptp** library API reference entry.

   ```
   -> ptpdAdd "gei1",2,1,1,0 ,128,-3,1
   -> ptpdStart
   ```

4. On the PTP slave node, bind the PTP daemon to Ethernet interface **gei1** and start it.

   Bind the PTP daemon as a slave clock, with PTP sync request intervals of 8 times a second using the IEEE802.1AS PTP protocol.

   ```
   -> ptpdAdd "gei1",2,0,1,0 ,128,-3,1
   -> ptpdStart
   ```

5. Wait for the debug messages on your slave device console to display synchronization messages before continuing with calls to the TSN clock.
6. Set debug level to **2** on **gei1** on both devices.

```
-> ptpdSetDebugLevel "gei1", 1, 2
```

7. On the master node, execute the demo code to initialize the timer demo, allocate a timer, and enable a timer.

   For example:

```
-> usrTsnClockIntReroute "gei",1,3 (optional)
-> usrTsnClockAlloc "gei",1,0,1000
-> usrTsnClockEnable "gei",1,0,2,0
```

   The parameters for the functions are:

   usrTsnClockIntReroute( )

   A pointer to the Ethernet device name, the Ethernet device index, the CPU core index (0 based), This step is optional and only used for the performance optimization.

   usrTsnClockAlloc( )

   A pointer to the Ethernet device name, the Ethernet device index, the 1588 clock timer number, and the number of ticks per second (for a one-shot interrupt, you would set this to zero).

   usrTsnClockEnable( )

   A pointer to the Ethernet device name, the Ethernet device index, the 1588 clock timer number, the offset to the current second, and the absolute start nanosecond.

8. On the slave node, execute the same commands.

```
-> usrTsnClockIntReroute "gei",1,3 (optional)
-> usrTsnClockAlloc "gei",1,0,1000
-> usrTsnClockEnable "gei",1,0,2,0
```

9. To see the TSN clock synchronization, attach an oscilloscope to the SDP0 of the Intel I210 network controller and see that the pins toggle synchronously.

Next Steps

The demo also includes functions for disabling and releasing TSN timers. For example, once you have initialized, allocated and enabled a TSN clock, you can disable and re-enable it as follows:

```
-> usrTsnClockDisable "gei",1,0
-> usrTsnClockEnable "gei",1,0,3,0
```

And, you can release and re-instantiate a clock like this:

```
-> usrTsnClockRelease "gei",1,0
-> usrTsnClockAlloc "gei",1,0,1000
-> usrTsnClockEnable "gei",1,0,2,0
```

11. LEGACY QAV_QBV SUPPORT

1. Legacy Qav_Qbv Support

VxWorks includes a set of legacy APIs for IEEE 802.1Qbv support not integrated with the network stack.

The preferred way to send time-sensitive networking (TSN) traffic is, however, using TSN streams.

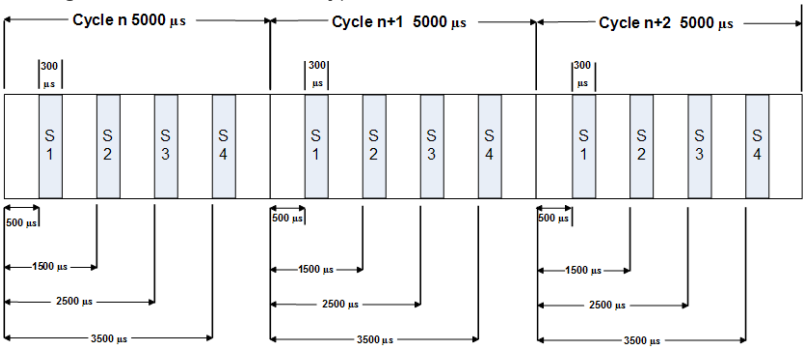
2. Qav_Qbv Library

The Qav_Qbv TSN library provide an application program interface (API) for transmitting scheduled traffic. The API calculates the launch time and transfers the raw frame to the network device driver.

Function	Description
tsnQbvTimerCycleCreate( )	Create a periodic timer cycle for a QBV instance that is bound to a special Ethernet physical device. If the parameter cycleStart is less than the 1588 system time, the cycle starts at the next system time boundary with a count of zero nanoseconds.
tsnQbvTimeSlotAdd( )	Add a time slot to the Qbv timer cycle.
tsnQbvTimeSlotDelete( )	Delete a time slot from the Qbv timer cycle.
tsnQbvMuxSend( )	Send an Ethernet packet with the Qbv mux.

Note that this API operates outside of the network stack and therefore the user must create the full frame, including all the protocol headers.

The figure below illustrates a typical TSN traffic schedule that defines four timeslots in a repeating cycle.



The first timeslot begins 500 microseconds into the cycle and lasts for 300 microseconds. Each timeslot is 300-microseconds long. The second, third and fourth slots begin at 1500, 2500 and 3500 microseconds into the schedule. The schedule repeats every 5000 microseconds. The example below shows how to create the traffic schedule above using the API:

```
cycleId = tsnQbvTimerCycleCreate ("gei",0,&ts,5000);
if (cycleId != -1)
```

```
{
tsnQbvTimeSlotAdd(cycleId, 1, 500, 300);
tsnQbvTimeSlotAdd(cycleId, 2, 1500, 300);
tsnQbvTimeSlotAdd(cycleId, 3, 2500, 300);
tsnQbvTimeSlotAdd(cycleId, 4, 3500, 300);
}
```

After the schedule is created, scheduled traffic is sent using the tsnQbvMuxSend( ) API. The example below shows a 128-byte frame in the third timeslot:

```
-> tsnQbvMuxSend(cycleId, 0, 3 , 0, 0, buf, 128);
```

For additional information, refer to the specific API documentation.